Introduction

Along these tutorials, we will see how **SCOPE** interacts with the different parts of the code to handle the execution of computational workflows.

These are the topics covered in each tutorial:

1. The **System** class and its sources: the **Specie**, **Cell** and **Atom** classes
2. The Computational workflow: **Branch**, **Workflow**, **Job**, and **Computation** classes
3. The **State** class
4. The **Data**, **Collection** and **VNM** classes
5. The **Input_data** class, and **scope input files**
6. Running **SCOPE** - Part 1: File Structure
7. Running **SCOPE** - Part 2: Execution
8. Running **SCOPE** - Part 3: Detailed Actions
9. The Azo add-on: Subclasses, Analysis and Creation of Azo Systems

Tutorial 4: Data, Collection and VNM Classes

In this tutorial, we will compute the thermodynamic properties for the "b3lpy_opt" **State** that we found in Tutorial 3, which includes the vibrational normal modes of a molecule. While doing so:

- we will discuss the **Data** and **Collection** classes, which are used in SCOPE to interact with -and store- results
- we will discuss the **VNM** class (VNM = Vibrational Normal Mode), which is currently the only quantum-chemistry-oriented class fully implemented in SCOPE

In [1]:
```
import os
import scope
import numpy as np
```

In [2]:
```
## Path of the data folder. It should be "os.path.abspath('.')+'/Data"
data_folder = os.path.abspath('../')+'/Data/1-Tutorial_1/'

## Loads the System object from a binary file, provided in the tutorial fold
sys = scope.load_binary(f"{data_folder}/Systems/ABITEM/ABITEM.npy")
```

Loading [MathJax]/extensions/Safe.js

```
## This will set the main path of the system to the data folder, so that any
sys.set_main_path(data_folder)
```

Out[2]: True

In [3]:
```
## Then, we select the source
found, source = sys.find_source("ref_hs_mol")

## And the state
found, final_state = source.find_state("b3lyp_opt")
print(final_state)
```

```
--------------------------------
------ SCOPE STATE Object ------
--------------------------------
 Name                  = b3lyp_opt
 Source Name           = ref_hs_mol
 Source Type           = specie
 Labels                = Fe...
 Coord                 = [3.774774, 3.779086, 13.494815]...
 Number of Units (Z)   = 1

 Has VNMs              = YES
 Is Minimum            = True
 Is a Transition State = False
 First Freq (cm-1)     = 21.6469

 Num of Molecules:     = 1
 With Formulae:
    0: H18-C20-N6-S2-Fe
```

Vibrational Normal Modes (VNMs)

In [4]:
```
## Notice that the state is a minimum energy structure:
print(final_state.isminimum)
```

```
True
```

In [5]:
```
## This is because the user provided the VNM. Here's the first one
print(final_state.VNMs[0])
```

```
----------------------------------
------   SCOPE VNM Object    -----
----------------------------------
 Index                  = 1
 Freq (cm-1)            = 21.6469
 IR Intensity (KM/Mole) = 1.0847
 Reduced Mass (AMU)     = 14.2624
 Has Mode               = False
```

In [6]:
```
# By default, the eigenvectors of the VNMs are not stored...
print(final_state.VNMs[0].has_mode)
```

Loading [MathJax]/extensions/Safe.js

```
False
```

In [7]:
```
# ... but they can be parsed if needed. To do it:
freq_comp = final_state.computations[-1]

# And parse the frequencies
from scope.register_data import reg_frequencies
worked = reg_frequencies(freq_comp, witheigen=True)
print(final_state.VNMs[0].has_mode)
```

```
True
```

Thermochemistry

Data Structure: Data and Collection

In [8]:
```
final_state
```

Out[8]:
```
--------------------------------
------ SCOPE STATE Object ------
--------------------------------
 Name                  = b3lyp_opt
 Source Name           = ref_hs_mol
 Source Type           = specie
 Labels                = Fe...
 Coord                 = [3.774774, 3.779086, 13.494815]...
 Number of Units (Z)   = 1

 Has VNMs              = YES
 Is Minimum            = True
 Is a Transition State = False
 First Freq (cm-1)     = 21.6469

 Num of Molecules:     = 1
 With Formulae:
    0: H18-C20-N6-S2-Fe
```

In [9]:
```
# Because the state contains the VNMs, we can compute the Thermodynamic Prop
final_state.get_thermal_data(temp=298.15)
```

In [10]:
```
# The above function computes the vibrational enthalpy (Hvib) and entropy (S
# These, together with the electronic enthalpy (Helec) and entropy (Selec) a
print(final_state.results["Helec"])
print(final_state.results["Selec"])
```

```
Helec: -3159.62579289 au
Selec:   0.00000510 au
```

In [11]:
```
# Notice that Helec and Selec are just a single instance of a so called DATA
print(final_state.results["Helec"].object_type)

# The DATA class forces the results to be stored together with the units, an
print(final_state.results["Helec"].value)
print(final_state.results["Helec"].units)
l_state.results["Helec"].function)
```

Loading [MathJax]/extensions/Safe.js

```
data
-3159.62579289
au
state.get_thermal_data()
```

In [12]:
```
# In turn, Hvib, Svib and Gtot are a so-called COLLECTION class. This class
# For instance, let's inspect Gtot:
Gtot = final_state.results["Gtot"]
print(Gtot)
```

```
----------------------------------------------------
   COLLECTION OF DATA   = Gtot
----------------------------------------------------
 Variable              = temperature
 Num of Entries        = 1
 First                 = -3159.3011019824903
 Last                  = -3159.3011019824903
 Units                 = au
 Function              = state.get_thermal_data()
```

In [13]:
```
## Collections imposes a certain structure on data. Gtot is a collection, be
Gtot.variable
```

Out[13]: 'temperature'

In [14]:
```
# By default, if a new temperature is requested, it is computed and added to
final_state.get_thermal_data(temp=10)
final_state.get_thermal_data(temp=20)

# Notice that, now, Gtot has 3 values (see Num of Entries below).
print(Gtot)
```

```
----------------------------------------------------
   COLLECTION OF DATA   = Gtot
----------------------------------------------------
 Variable              = temperature
 Num of Entries        = 3
 First                 = -3159.3011019824903
 Last                  = -3159.274027677308
 Units                 = au
 Function              = state.get_thermal_data()
```

In [15]:
```
# The input variable 'temp' can be a single numeric value (int, float), a li
# The default value is room temperature = 298.15K
final_state.get_thermal_data(temp=[10, 20], overwrite=True)
final_state.get_thermal_data(temp=range(10,500,1), overwrite=True)

# When overwrite=True, the Collection is re-computed, which implies that new
# For the analyses below, we stick to the last execution
Gtot = final_state.results["Gtot"]     ## Because Gtot has been overwritten,
print(Gtot)
```

Loading [MathJax]/extensions/Safe.js

```
-----------------------------------------------------
   COLLECTION OF DATA   = Gtot
-----------------------------------------------------
 Variable              = temperature
 Num of Entries        = 490
 First                 = -3159.273935896281
 Last                  = -3159.345634045812
 Units                 = au
 Function              = state.get_thermal_data()
```

In [16]:
```
# To retrieve the values out of a COLLECTION class, you can:
print(final_state.results["Gtot"].get_values()[0:10]) # Notice that only the
```

```
[-3159.2739359  -3159.27394256 -3159.2739497  -3159.27395736
 -3159.27396557 -3159.27397437 -3159.27398376 -3159.27399377
 -3159.27400442 -3159.27401572]
```

In [17]:
```
# To access an individual entry, you can search by the name of variable, and
final_state.results["Gtot"].find_value_with_property("temperature", 200)
```

Out[17]:
```
Gtot: -3159.28665610 au (at temperature=200)
```

PV term

In [18]:
```
## When the source of the state is a Cell, the PV term is not computed by de
## The reason is that, normally, is very small. However, it can be evaluated
final_state.compute_PV_term(pressure=101.325)

## Here, because the source is a molecule, the PV term is zero
```

Out[18]:
```
PV:   0.00000000 kj
```

Subtracting and Plotting Collections

In [19]:
```
# Collections can be subtracted:
HS_state = final_state # we were operating with the HS molecule's data

## Lets select the LS as well, and compute its TD properties
found, source   = sys.find_source("ref_ls_mol")
found, LS_state = source.find_state("b3lyp_opt")
LS_state.get_thermal_data(temp=range(10,500,1))

print(HS_state)
print(LS_state)
```

Loading [MathJax]/extensions/Safe.js

```
--------------------------------
------ SCOPE STATE Object ------
--------------------------------
 Name                  = b3lyp_opt
 Source Name           = ref_hs_mol
 Source Type           = specie
 Labels                = Fe...
 Coord                 = [3.774774, 3.779086, 13.494815]...
 Number of Units (Z)   = 1

 Has VNMs              = YES
 Is Minimum            = True
 Is a Transition State = False
 First Freq (cm-1)     = 21.6469

 Num of Molecules:     = 1
 With Formulae:
    0: H18-C20-N6-S2-Fe

--------------------------------
------ SCOPE STATE Object ------
--------------------------------
 Name                  = b3lyp_opt
 Source Name           = ref_ls_mol
 Source Type           = specie
 Labels                = Fe...
 Coord                 = [4.171778, 7.781139, 4.158969]...
 Number of Units (Z)   = 1

 Has VNMs              = YES
 Is Minimum            = True
 Is a Transition State = False
 First Freq (cm-1)     = 24.4354

 Num of Molecules:     = 1
 With Formulae:
    0: H18-C20-N6-S2-Fe
```

In [20]:

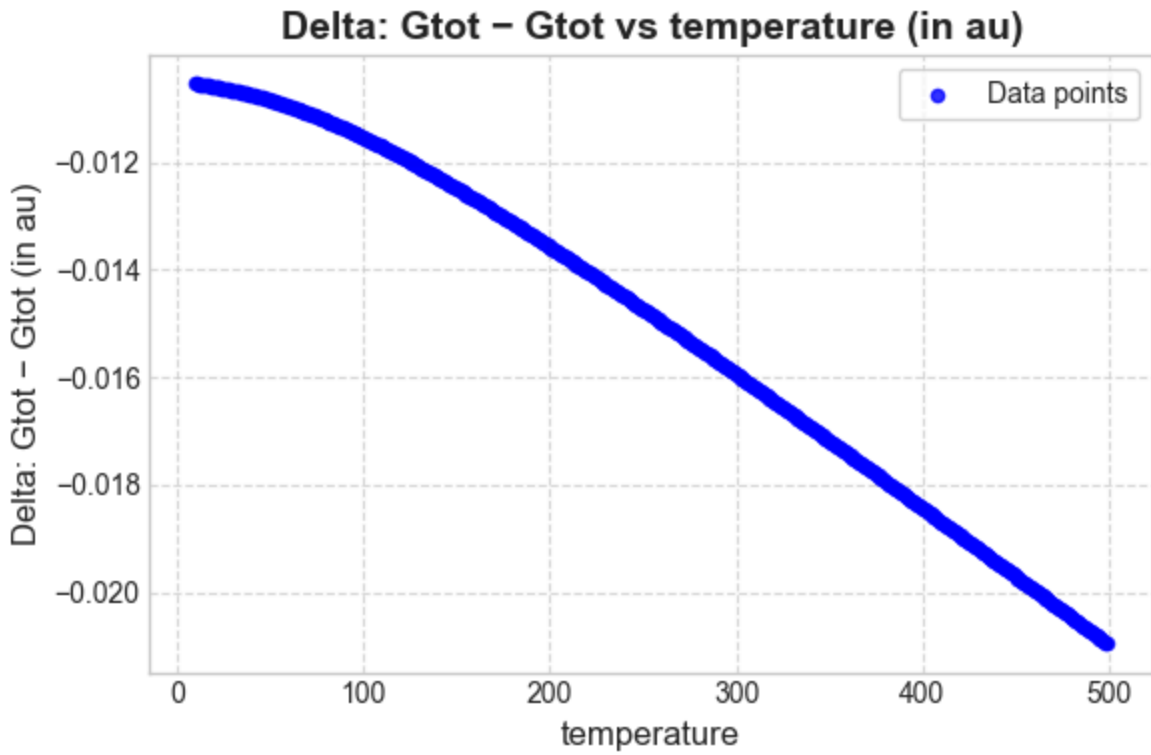
```python
# Collections to substract
HS_Gtot = HS_state.results["Gtot"]
LS_Gtot = LS_state.results["Gtot"]

# New collection:
dGtot = HS_Gtot - LS_Gtot

# There is a simple function to plot Collections
dGtot.plot()

# If the two substracting collection have different data (for instance, diff
# The same philosophy applies to summing Collections, which would be execute
```

Loading [MathJax]/extensions/Safe.js



Computation of T12 (SCO addon)

In [21]:
```
from scope_sco.sco_functions import get_T12

## For SCO systems, the transition temperature associated with the thermal s

## The branch is the platform where the results will be stored
found, branch = sys.find_branch("Isolated")
## The T12 is computed using the values stored in the two relevant states.
worked, t12 = get_T12(branch, HS_state, LS_state, flexible=True)
print(worked)
print(t12)
# In this case, there is no T12, since the HS state is always more stable th
```

```
True
T12: None
```

Actual Values (Accuracy)

In [22]:
```
## Above, we have seen how the computed thermochemistry values are handled b
```

In [23]:
```
## Helec, is just the "SCF Done" energy in Gaussian16:
lines       = scope.parse_general.read_lines_file(freq_comp.out_path)
line, found = scope.parse_general.search_string("SCF Done", lines, typ='last
print(lines[line].strip())
print(final_state.results["Helec"])

## Selec, is obtained as R·ln(2S+1), so:
print(f"Selec:   {scope.constants.R_J*np.log(final_state.spin_multiplicity)/
print(final_state.results["Selec"])
```

Loading [MathJax]/extensions/Safe.js

```
SCF Done:  E(UB3LYP) =  -3159.62579289     A.U. after   39 cycles
Helec: -3159.62579289 au
Selec:   0.00000510 au
Selec:   0.00000510 au
```

In [24]:
```
## Hvib is computed by the get_Hvib function. Here, we do it at 298.15 to be
hvib = scope.thermodynamics.get_Hvib(final_state.freqs_cm, temp=298.15, freq
print(hvib)

## Value in G16:
line, found = scope.parse_general.search_string("Thermal correction to Energ
print(lines[line].strip())

## The difference between the two terms (0.378882 vs. 0.3760) corresponds to
```

```
Hvib:   0.37604956 au (at temperature=298.15)
Thermal correction to Energy=                    0.378882
```

In [25]:
```
## In debug mode, you can compare the contributions of each VNM, with the va
scope.thermodynamics.get_Hvib(final_state.freqs_cm, temp=298.15, freq_units=
```

Loading [MathJax]/extensions/Safe.js

```
GET_Hvib: Computing Hvib with 135 frequencies, and first: 21.6469 cm
        0 i= 21.65 cm-1: Hvib(i)= 9.4505e-04 a.u, Hvib(i)= 0.5930 kcal/mol
        1 i= 28.41 cm-1: Hvib(i)= 9.4567e-04 a.u, Hvib(i)= 0.5934 kcal/mol
        2 i= 29.16 cm-1: Hvib(i)= 9.4575e-04 a.u, Hvib(i)= 0.5935 kcal/mol
        3 i= 34.17 cm-1: Hvib(i)= 9.4633e-04 a.u, Hvib(i)= 0.5938 kcal/mol
        4 i= 39.76 cm-1: Hvib(i)= 9.4708e-04 a.u, Hvib(i)= 0.5943 kcal/mol
        5 i= 57.09 cm-1: Hvib(i)= 9.5015e-04 a.u, Hvib(i)= 0.5962 kcal/mol
        6 i= 70.07 cm-1: Hvib(i)= 9.5317e-04 a.u, Hvib(i)= 0.5981 kcal/mol
        7 i= 74.84 cm-1: Hvib(i)= 9.5443e-04 a.u, Hvib(i)= 0.5989 kcal/mol
        8 i= 80.77 cm-1: Hvib(i)= 9.5611e-04 a.u, Hvib(i)= 0.6000 kcal/mol
        9 i= 90.43 cm-1: Hvib(i)= 9.5913e-04 a.u, Hvib(i)= 0.6019 kcal/mol
        10 i= 94.48 cm-1: Hvib(i)= 9.6049e-04 a.u, Hvib(i)= 0.6027 kcal/mol
        11 i= 107.83 cm-1: Hvib(i)= 9.6540e-04 a.u, Hvib(i)= 0.6058 kcal/mol
        12 i= 123.19 cm-1: Hvib(i)= 9.7183e-04 a.u, Hvib(i)= 0.6098 kcal/mol
        13 i= 137.79 cm-1: Hvib(i)= 9.7872e-04 a.u, Hvib(i)= 0.6142 kcal/mol
        14 i= 154.67 cm-1: Hvib(i)= 9.8762e-04 a.u, Hvib(i)= 0.6197 kcal/mol
        15 i= 168.08 cm-1: Hvib(i)= 9.9539e-04 a.u, Hvib(i)= 0.6246 kcal/mol
        16 i= 170.21 cm-1: Hvib(i)= 9.9669e-04 a.u, Hvib(i)= 0.6254 kcal/mol
        17 i= 180.54 cm-1: Hvib(i)= 1.0032e-03 a.u, Hvib(i)= 0.6295 kcal/mol
        18 i= 214.30 cm-1: Hvib(i)= 1.0269e-03 a.u, Hvib(i)= 0.6444 kcal/mol
        19 i= 218.59 cm-1: Hvib(i)= 1.0302e-03 a.u, Hvib(i)= 0.6464 kcal/mol
        20 i= 227.35 cm-1: Hvib(i)= 1.0370e-03 a.u, Hvib(i)= 0.6508 kcal/mol
        21 i= 231.52 cm-1: Hvib(i)= 1.0404e-03 a.u, Hvib(i)= 0.6529 kcal/mol
        22 i= 252.15 cm-1: Hvib(i)= 1.0579e-03 a.u, Hvib(i)= 0.6638 kcal/mol
        23 i= 281.74 cm-1: Hvib(i)= 1.0853e-03 a.u, Hvib(i)= 0.6811 kcal/mol
        24 i= 298.90 cm-1: Hvib(i)= 1.1025e-03 a.u, Hvib(i)= 0.6918 kcal/mol
        25 i= 341.99 cm-1: Hvib(i)= 1.1493e-03 a.u, Hvib(i)= 0.7212 kcal/mol
        26 i= 347.29 cm-1: Hvib(i)= 1.1555e-03 a.u, Hvib(i)= 0.7251 kcal/mol
        27 i= 374.27 cm-1: Hvib(i)= 1.1879e-03 a.u, Hvib(i)= 0.7454 kcal/mol
        28 i= 411.96 cm-1: Hvib(i)= 1.2364e-03 a.u, Hvib(i)= 0.7759 kcal/mol
        29 i= 419.45 cm-1: Hvib(i)= 1.2465e-03 a.u, Hvib(i)= 0.7822 kcal/mol
        30 i= 428.93 cm-1: Hvib(i)= 1.2594e-03 a.u, Hvib(i)= 0.7903 kcal/mol
        31 i= 438.16 cm-1: Hvib(i)= 1.2723e-03 a.u, Hvib(i)= 0.7984 kcal/mol
        32 i= 441.06 cm-1: Hvib(i)= 1.2763e-03 a.u, Hvib(i)= 0.8009 kcal/mol
        33 i= 477.38 cm-1: Hvib(i)= 1.3289e-03 a.u, Hvib(i)= 0.8339 kcal/mol
        34 i= 479.18 cm-1: Hvib(i)= 1.3316e-03 a.u, Hvib(i)= 0.8356 kcal/mol
        35 i= 487.21 cm-1: Hvib(i)= 1.3437e-03 a.u, Hvib(i)= 0.8432 kcal/mol
        36 i= 489.46 cm-1: Hvib(i)= 1.3471e-03 a.u, Hvib(i)= 0.8453 kcal/mol
        37 i= 499.51 cm-1: Hvib(i)= 1.3624e-03 a.u, Hvib(i)= 0.8549 kcal/mol
        38 i= 500.67 cm-1: Hvib(i)= 1.3642e-03 a.u, Hvib(i)= 0.8561 kcal/mol
        39 i= 513.73 cm-1: Hvib(i)= 1.3845e-03 a.u, Hvib(i)= 0.8688 kcal/mol
        40 i= 520.49 cm-1: Hvib(i)= 1.3951e-03 a.u, Hvib(i)= 0.8755 kcal/mol
        41 i= 623.06 cm-1: Hvib(i)= 1.5671e-03 a.u, Hvib(i)= 0.9834 kcal/mol
        42 i= 637.25 cm-1: Hvib(i)= 1.5923e-03 a.u, Hvib(i)= 0.9992 kcal/mol
        43 i= 641.25 cm-1: Hvib(i)= 1.5995e-03 a.u, Hvib(i)= 1.0037 kcal/mol
        44 i= 648.08 cm-1: Hvib(i)= 1.6118e-03 a.u, Hvib(i)= 1.0114 kcal/mol
        45 i= 659.13 cm-1: Hvib(i)= 1.6318e-03 a.u, Hvib(i)= 1.0240 kcal/mol
        46 i= 676.74 cm-1: Hvib(i)= 1.6641e-03 a.u, Hvib(i)= 1.0442 kcal/mol
        47 i= 748.43 cm-1: Hvib(i)= 1.7997e-03 a.u, Hvib(i)= 1.1293 kcal/mol
        48 i= 751.22 cm-1: Hvib(i)= 1.8051e-03 a.u, Hvib(i)= 1.1327 kcal/mol
        49 i= 760.73 cm-1: Hvib(i)= 1.8236e-03 a.u, Hvib(i)= 1.1443 kcal/mol
        50 i= 767.76 cm-1: Hvib(i)= 1.8373e-03 a.u, Hvib(i)= 1.1529 kcal/mol
        51 i= 769.43 cm-1: Hvib(i)= 1.8406e-03 a.u, Hvib(i)= 1.1550 kcal/mol
        52 i= 770.97 cm-1: Hvib(i)= 1.8436e-03 a.u, Hvib(i)= 1.1569 kcal/mol
        53 i= 831.92 cm-1: Hvib(i)= 1.9649e-03 a.u, Hvib(i)= 1.2330 kcal/mol
        54 i= 837.73 cm-1: Hvib(i)= 1.9767e-03 a.u, Hvib(i)= 1.2404 kcal/mol
```

Loading [MathJax]/extensions/Safe.js

```
55 i= 858.95 cm-1: Hvib(i)= 2.0198e-03 a.u, Hvib(i)= 1.2675 kcal/mol
56 i= 866.48 cm-1: Hvib(i)= 2.0352e-03 a.u, Hvib(i)= 1.2771 kcal/mol
57 i= 882.12 cm-1: Hvib(i)= 2.0674e-03 a.u, Hvib(i)= 1.2973 kcal/mol
58 i= 900.44 cm-1: Hvib(i)= 2.1053e-03 a.u, Hvib(i)= 1.3211 kcal/mol
59 i= 902.14 cm-1: Hvib(i)= 2.1088e-03 a.u, Hvib(i)= 1.3233 kcal/mol
60 i= 904.92 cm-1: Hvib(i)= 2.1146e-03 a.u, Hvib(i)= 1.3269 kcal/mol
61 i= 925.96 cm-1: Hvib(i)= 2.1584e-03 a.u, Hvib(i)= 1.3544 kcal/mol
62 i= 960.66 cm-1: Hvib(i)= 2.2314e-03 a.u, Hvib(i)= 1.4002 kcal/mol
63 i= 973.02 cm-1: Hvib(i)= 2.2576e-03 a.u, Hvib(i)= 1.4166 kcal/mol
64 i= 985.19 cm-1: Hvib(i)= 2.2834e-03 a.u, Hvib(i)= 1.4329 kcal/mol
65 i= 992.25 cm-1: Hvib(i)= 2.2985e-03 a.u, Hvib(i)= 1.4423 kcal/mol
66 i= 995.50 cm-1: Hvib(i)= 2.3054e-03 a.u, Hvib(i)= 1.4467 kcal/mol
67 i= 1006.05 cm-1: Hvib(i)= 2.3279e-03 a.u, Hvib(i)= 1.4608 kcal/mo
l
68 i= 1019.70 cm-1: Hvib(i)= 2.3572e-03 a.u, Hvib(i)= 1.4792 kcal/mo
l
69 i= 1020.21 cm-1: Hvib(i)= 2.3583e-03 a.u, Hvib(i)= 1.4798 kcal/mo
l
70 i= 1021.20 cm-1: Hvib(i)= 2.3604e-03 a.u, Hvib(i)= 1.4812 kcal/mo
l
71 i= 1023.80 cm-1: Hvib(i)= 2.3660e-03 a.u, Hvib(i)= 1.4847 kcal/mo
l
72 i= 1029.62 cm-1: Hvib(i)= 2.3785e-03 a.u, Hvib(i)= 1.4925 kcal/mo
l
73 i= 1032.07 cm-1: Hvib(i)= 2.3838e-03 a.u, Hvib(i)= 1.4958 kcal/mo
l
74 i= 1066.23 cm-1: Hvib(i)= 2.4575e-03 a.u, Hvib(i)= 1.5421 kcal/mo
l
75 i= 1070.34 cm-1: Hvib(i)= 2.4664e-03 a.u, Hvib(i)= 1.5477 kcal/mo
l
76 i= 1071.85 cm-1: Hvib(i)= 2.4697e-03 a.u, Hvib(i)= 1.5498 kcal/mo
l
77 i= 1111.53 cm-1: Hvib(i)= 2.5561e-03 a.u, Hvib(i)= 1.6040 kcal/mo
l
78 i= 1113.91 cm-1: Hvib(i)= 2.5613e-03 a.u, Hvib(i)= 1.6072 kcal/mo
l
79 i= 1117.18 cm-1: Hvib(i)= 2.5684e-03 a.u, Hvib(i)= 1.6117 kcal/mo
l
80 i= 1151.20 cm-1: Hvib(i)= 2.6430e-03 a.u, Hvib(i)= 1.6585 kcal/mo
l
81 i= 1153.03 cm-1: Hvib(i)= 2.6470e-03 a.u, Hvib(i)= 1.6610 kcal/mo
l
82 i= 1156.06 cm-1: Hvib(i)= 2.6537e-03 a.u, Hvib(i)= 1.6652 kcal/mo
l
83 i= 1157.24 cm-1: Hvib(i)= 2.6563e-03 a.u, Hvib(i)= 1.6668 kcal/mo
l
84 i= 1158.00 cm-1: Hvib(i)= 2.6579e-03 a.u, Hvib(i)= 1.6679 kcal/mo
l
85 i= 1223.36 cm-1: Hvib(i)= 2.8023e-03 a.u, Hvib(i)= 1.7584 kcal/mo
l
86 i= 1231.69 cm-1: Hvib(i)= 2.8208e-03 a.u, Hvib(i)= 1.7701 kcal/mo
l
87 i= 1241.77 cm-1: Hvib(i)= 2.8431e-03 a.u, Hvib(i)= 1.7841 kcal/mo
l
88 i= 1255.12 cm-1: Hvib(i)= 2.8728e-03 a.u, Hvib(i)= 1.8027 kcal/mo
l
```

Loading [MathJax]/extensions/Safe.js

```
        89 i= 1284.35 cm-1: Hvib(i)= 2.9379e-03 a.u, Hvib(i)= 1.8436 kcal/mo
l
        90 i= 1302.35 cm-1: Hvib(i)= 2.9781e-03 a.u, Hvib(i)= 1.8688 kcal/mo
l
        91 i= 1305.07 cm-1: Hvib(i)= 2.9841e-03 a.u, Hvib(i)= 1.8726 kcal/mo
l
        92 i= 1310.85 cm-1: Hvib(i)= 2.9970e-03 a.u, Hvib(i)= 1.8807 kcal/mo
l
        93 i= 1319.78 cm-1: Hvib(i)= 3.0170e-03 a.u, Hvib(i)= 1.8932 kcal/mo
l
        94 i= 1326.96 cm-1: Hvib(i)= 3.0331e-03 a.u, Hvib(i)= 1.9033 kcal/mo
l
        95 i= 1337.03 cm-1: Hvib(i)= 3.0556e-03 a.u, Hvib(i)= 1.9174 kcal/mo
l
        96 i= 1346.99 cm-1: Hvib(i)= 3.0779e-03 a.u, Hvib(i)= 1.9314 kcal/mo
l
        97 i= 1375.19 cm-1: Hvib(i)= 3.1411e-03 a.u, Hvib(i)= 1.9711 kcal/mo
l
        98 i= 1389.64 cm-1: Hvib(i)= 3.1736e-03 a.u, Hvib(i)= 1.9915 kcal/mo
l
        99 i= 1409.68 cm-1: Hvib(i)= 3.2186e-03 a.u, Hvib(i)= 2.0197 kcal/mo
l
        100 i= 1437.74 cm-1: Hvib(i)= 3.2818e-03 a.u, Hvib(i)= 2.0593 kcal/m
ol
        101 i= 1449.23 cm-1: Hvib(i)= 3.3077e-03 a.u, Hvib(i)= 2.0756 kcal/m
ol
        102 i= 1460.78 cm-1: Hvib(i)= 3.3337e-03 a.u, Hvib(i)= 2.0919 kcal/m
ol
        103 i= 1462.36 cm-1: Hvib(i)= 3.3372e-03 a.u, Hvib(i)= 2.0941 kcal/m
ol
        104 i= 1462.40 cm-1: Hvib(i)= 3.3373e-03 a.u, Hvib(i)= 2.0942 kcal/m
ol
        105 i= 1475.77 cm-1: Hvib(i)= 3.3675e-03 a.u, Hvib(i)= 2.1131 kcal/m
ol
        106 i= 1498.60 cm-1: Hvib(i)= 3.4190e-03 a.u, Hvib(i)= 2.1455 kcal/m
ol
        107 i= 1502.60 cm-1: Hvib(i)= 3.4280e-03 a.u, Hvib(i)= 2.1511 kcal/m
ol
        108 i= 1505.47 cm-1: Hvib(i)= 3.4345e-03 a.u, Hvib(i)= 2.1552 kcal/m
ol
        109 i= 1618.77 cm-1: Hvib(i)= 3.6908e-03 a.u, Hvib(i)= 2.3160 kcal/m
ol
        110 i= 1619.64 cm-1: Hvib(i)= 3.6928e-03 a.u, Hvib(i)= 2.3173 kcal/m
ol
        111 i= 1621.88 cm-1: Hvib(i)= 3.6979e-03 a.u, Hvib(i)= 2.3204 kcal/m
ol
        112 i= 1642.35 cm-1: Hvib(i)= 3.7442e-03 a.u, Hvib(i)= 2.3496 kcal/m
ol
        113 i= 1645.77 cm-1: Hvib(i)= 3.7520e-03 a.u, Hvib(i)= 2.3544 kcal/m
ol
        114 i= 1647.30 cm-1: Hvib(i)= 3.7555e-03 a.u, Hvib(i)= 2.3566 kcal/m
ol
        115 i= 2107.86 cm-1: Hvib(i)= 4.8024e-03 a.u, Hvib(i)= 3.0136 kcal/m
ol
        116 i= 2140.00 cm-1: Hvib(i)= 4.8756e-03 a.u, Hvib(i)= 3.0595 kcal/m
ol
```

Loading [MathJax]/extensions/Safe.js

```
        117 i= 2968.89 cm-1: Hvib(i)= 6.7636e-03 a.u, Hvib(i)= 4.2442 kcal/m
ol
        118 i= 2975.05 cm-1: Hvib(i)= 6.7777e-03 a.u, Hvib(i)= 4.2531 kcal/m
ol
        119 i= 2982.33 cm-1: Hvib(i)= 6.7942e-03 a.u, Hvib(i)= 4.2635 kcal/m
ol
        120 i= 3027.29 cm-1: Hvib(i)= 6.8967e-03 a.u, Hvib(i)= 4.3277 kcal/m
ol
        121 i= 3045.11 cm-1: Hvib(i)= 6.9373e-03 a.u, Hvib(i)= 4.3532 kcal/m
ol
        122 i= 3059.78 cm-1: Hvib(i)= 6.9707e-03 a.u, Hvib(i)= 4.3742 kcal/m
ol
        123 i= 3161.05 cm-1: Hvib(i)= 7.2014e-03 a.u, Hvib(i)= 4.5189 kcal/m
ol
        124 i= 3164.23 cm-1: Hvib(i)= 7.2087e-03 a.u, Hvib(i)= 4.5235 kcal/m
ol
        125 i= 3166.47 cm-1: Hvib(i)= 7.2137e-03 a.u, Hvib(i)= 4.5267 kcal/m
ol
        126 i= 3168.13 cm-1: Hvib(i)= 7.2175e-03 a.u, Hvib(i)= 4.5291 kcal/m
ol
        127 i= 3169.43 cm-1: Hvib(i)= 7.2205e-03 a.u, Hvib(i)= 4.5309 kcal/m
ol
        128 i= 3179.27 cm-1: Hvib(i)= 7.2429e-03 a.u, Hvib(i)= 4.5450 kcal/m
ol
        129 i= 3180.83 cm-1: Hvib(i)= 7.2465e-03 a.u, Hvib(i)= 4.5472 kcal/m
ol
        130 i= 3181.46 cm-1: Hvib(i)= 7.2479e-03 a.u, Hvib(i)= 4.5481 kcal/m
ol
        131 i= 3186.70 cm-1: Hvib(i)= 7.2598e-03 a.u, Hvib(i)= 4.5556 kcal/m
ol
        132 i= 3195.05 cm-1: Hvib(i)= 7.2789e-03 a.u, Hvib(i)= 4.5676 kcal/m
ol
        133 i= 3195.63 cm-1: Hvib(i)= 7.2802e-03 a.u, Hvib(i)= 4.5684 kcal/m
ol
        134 i= 3196.98 cm-1: Hvib(i)= 7.2833e-03 a.u, Hvib(i)= 4.5703 kcal/m
ol
```

Out[25]:

```
Hvib:   0.37604956 au (at temperature=298.15)
```

In [26]:

```
## You can see above that, for i=21.65, corresponding to VNM number 1, Hvib(

## In the G16 output:
line, found = scope.parse_general.search_string("Vibration     1 ", lines, t

## You can find this block, where you can see that Hvib(i) is the same
print(lines[line-7].rstrip())
print(lines[line-6].rstrip())
print(lines[line].rstrip())
```

```
                    E (Thermal)             CV                S
                     KCal/Mol        Cal/Mol-Kelvin    Cal/Mol-Kelvin
Vibration     1          0.593              1.985              6.477
```

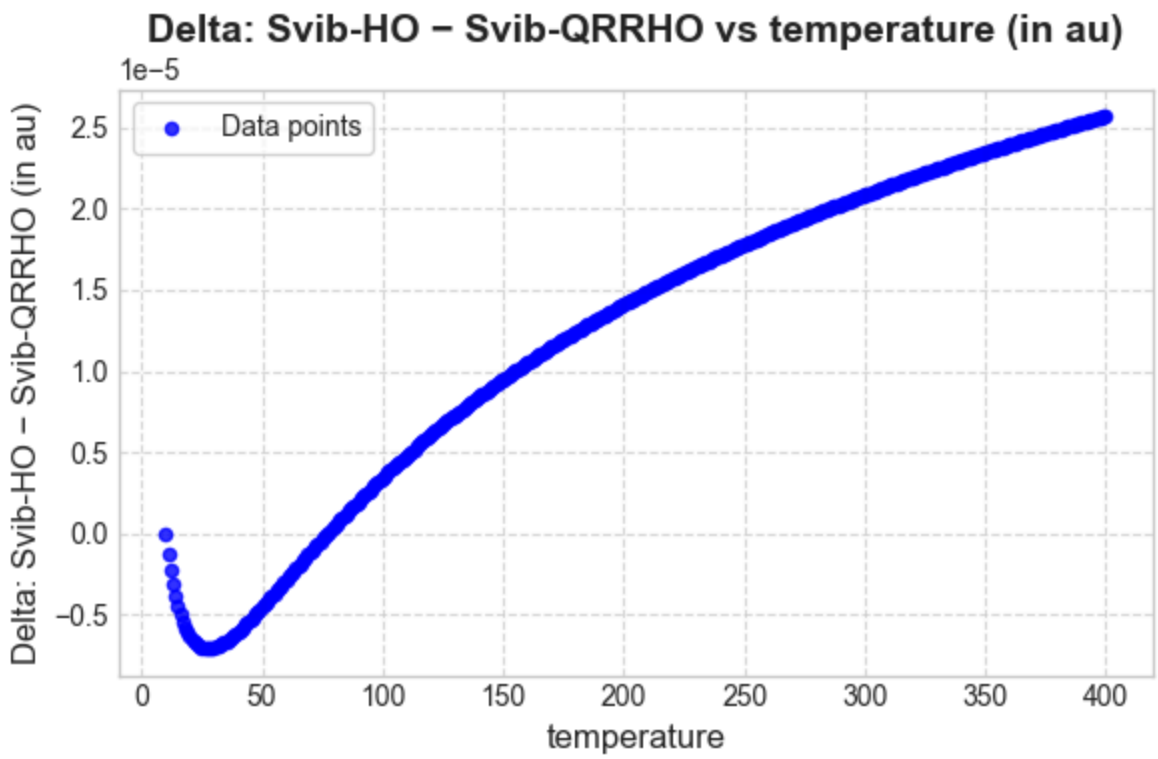
In [27]:

```
## For Svib, we implemented:
    # - The default Harmonic-Oscillator (HO) model
    # - The Quasi-Rigid-Rotor-Harmonic-Oscillator (QRRHO) model by Grimme, a
```

Loading [MathJax]/extensions/Safe.js

```
## It is possible to compare these methods:
vals1 = scope.classes_data.Collection('Svib-HO', 'temperature')
vals2 = scope.classes_data.Collection('Svib-QRRHO', 'temperature')
for t in range(10,401,1):
    vals1.add_data(scope.thermodynamics.get_Svib(final_state.freqs_cm, temp=
    vals2.add_data(scope.thermodynamics.get_Svib(final_state.freqs_cm, temp=

## The difference between those methods can be significant
(vals1 - vals2).plot()
```



In [28]:
```
## By default, the HO is employed. At 298.15 K:
val = scope.thermodynamics.get_Svib(final_state.freqs_cm, temp=298.15, freq_
print(val)
```

```
Svib:   0.00016716 au (at temperature=298.15)
```

In [29]:
```
## Summing the electronic entropy:
Stot = val.value + final_state.results["Selec"].value
## And multiplyhing by T
Stot = Stot * 298.15
## We obtain:
print(f"{Stot:.6f}")
```

```
0.051359
```

In [30]:
```
## Together with hvib estimated above at 298.15 K:
Gtot = hvib.value-Stot
print(f"{Gtot:.6f}")
```

```
0.324691
```

In [31]:
```
## The total thermal correction computed by Gaussian16 (ca. 0.290320 au)
d = scope.parse_general.search_string("Thermal correction to Gibbs
```

Loading [MathJax]/extensions/Safe.js

```
print(lines[line].strip())
```

```
Thermal correction to Gibbs Free Energy=         0.290320
```

In [32]:
```
## The difference between those two approaches (0.3247 vs 0.2903 a.u) is tha
```

Units Conversion

In [33]:
```
## A very basic system to change units has been implemented at the DATA clas
print(final_state.results["Helec"])
print('')

## print_in_units() will preserve the original units:
print(final_state.results["Helec"].print_in_units("ev"))
print(final_state.results["Helec"])
print('')

## Convert to units function will overwrite the DATA units permanently:
print(final_state.results["Helec"].convert_to_units("cm"))
print(final_state.results["Helec"])
```

```
Helec: -3159.62579289 au

Helec: -85977.83814102 ev
Helec: -3159.62579289 au

Helec: -693457607.04421556 cm
Helec: -693457607.04421556 cm
```

In [34]:
```
## Units can also be converted in Collections with:
vals1.convert_to_units('ev')
```

Out[34]:
```
-----------------------------------------------------
   COLLECTION OF DATA   = Svib-HO
-----------------------------------------------------
 Variable              = temperature
 Num of Entries        = 391
 First                 = 3.664615470396214e-05
 Last                  = 0.0059232533246649675
 Units                 = ev
 Function              = scope.thermodynamics.get_Svib()
```

Loading [MathJax]/extensions/Safe.js